BİL4010

Paralel programlama

6.Hafta

MPI Giriş

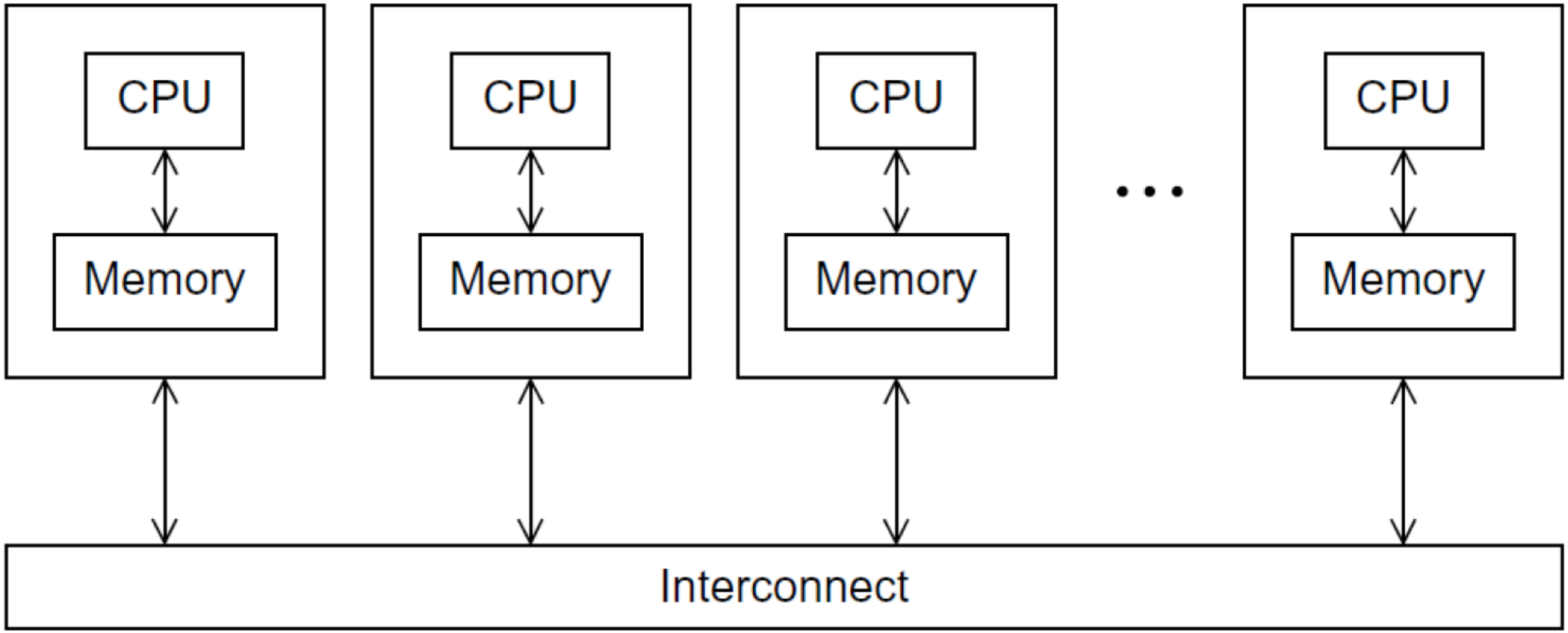
İçerik

- MPI ile Dağıtık Bellek Programlama
- MPI ile Hello World Uygulaması
- MPI Programını Derlemek
- MPI Programını Çalıştırmak
- MPI Fonksiyon ve Argümanları

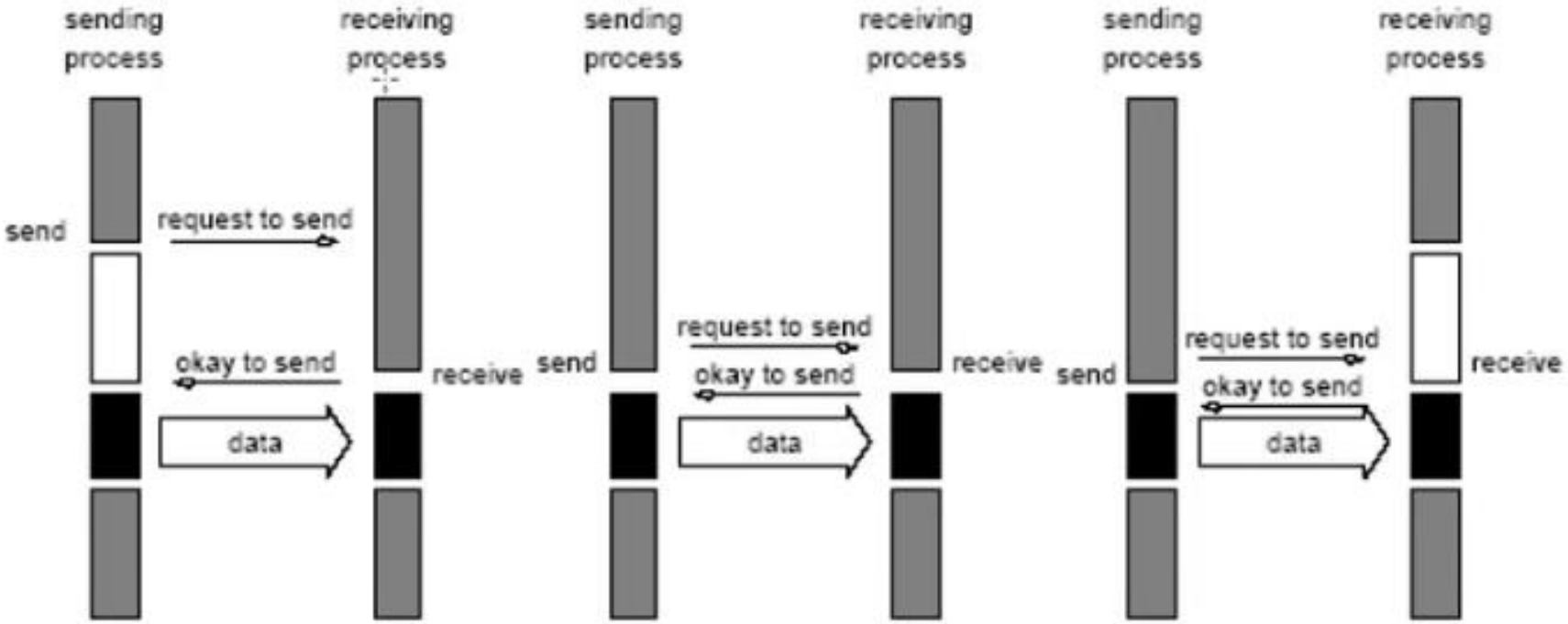
MPI ile Dağıtık Bellek Programlama

- Paralel yazılımlar iki alt başlıkta incelenir:
  - Hafıza paylaşımlı sistemler
  - Dağıtık hafızalı (bellekli) sistemler
- Bir programcı gözünden bakıldığında dağıtık bellekli sistemler çekirdek ve hafıza bütününden oluşan parçaların bir iletişim ağı ile haberleşerek bir bütünü oluşturmaktadır.
- Bu bölümde dağıtık bellekli sistemlerde mesaj-geçişi (message-passing) yöntemi ile nasıl paralel programlar geliştirildiğini incelenecektir.

Dağıtık Bellek Sistemi



Mesaj Aktarma İşlemleri (Non-Buffered Blocking)



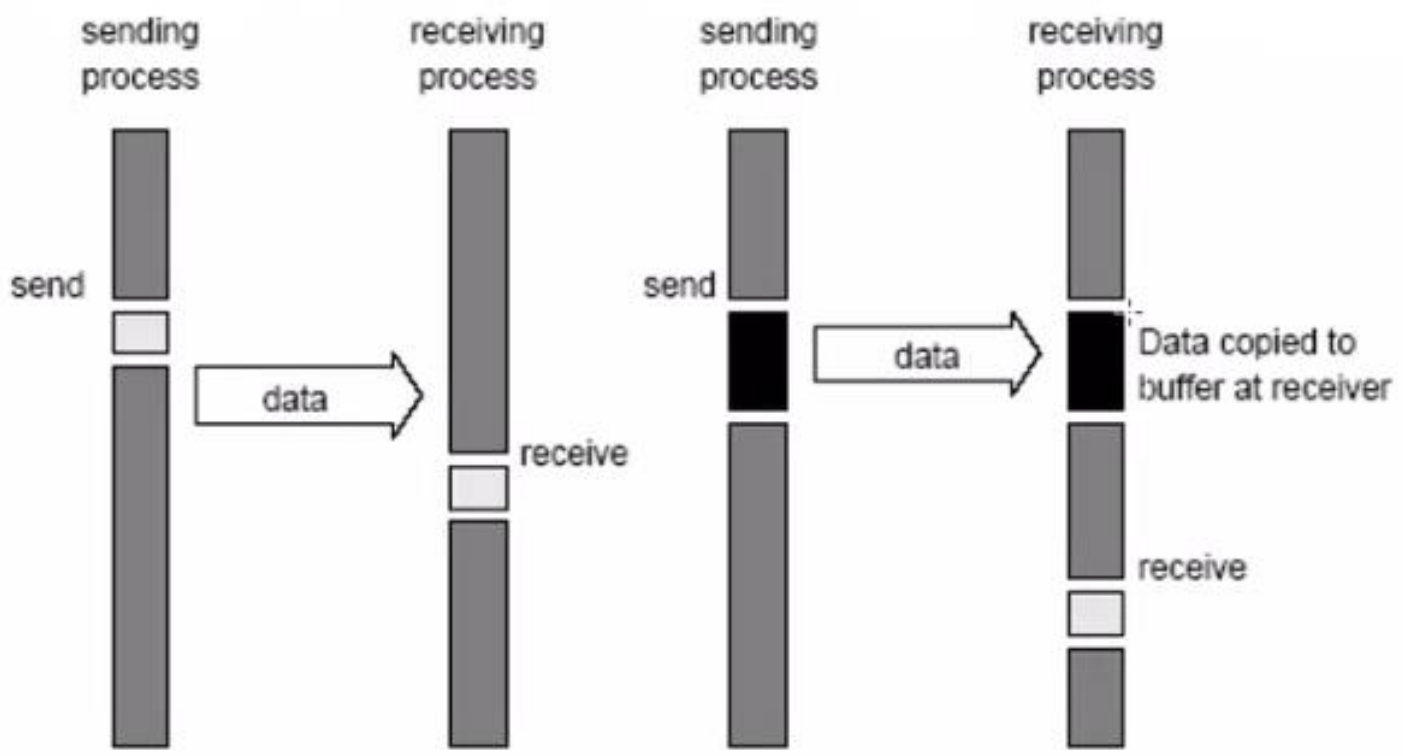
(a) Sender comes first; idling at sender

(b) Sender and receiver come at about the same time; idling minimized

(c) Receiver comes first; idling at receiver

Gönderenin ve alıcının benzer zamanlarda iletişim noktasına ulaşmadığı durumlarda, önemli miktarda bekleme masrafı olabileceği şekil (a) ve (c) de görülebilir.

Mesaj Aktarma İşlemleri (Buffered Blocking)



Arabelleğe aktarılan aktarım protokollerini engelleme: (a) gönderme ve alma uçlarında tamponlarla iletişim donanımının bulunduğu ortamda; (b) iletişim donanımı olmaması durumunda, gönderici, alıcıyı keser ve verileri alıcı ucunda arabellekte biriktirir.

MPI (Message-Passing Interface)

- Mesaj-geçişi programlarında (message-passing) sistem çekirdek-bellek bütününü tek bir işlem (process) gibi algılar.
- MPI sistemlerinde en az iki process arasında *send* ve *receive* fonksiyonları çalıştırılarak iletişim sağlanır.
- MPI yeni bir programlama dili değildir.
- MPI C/C++, Fortran veya Python dilleri tarafından kullanılabilen bir dağıtık bellek yönetimi kütüphanesidir.
- Bu kısımda MPI kütüphanesini ve fonksiyonlarını incelenecektir.

MPI Programını Visual Studio ile Derlemek

1) MPI SDK'larının yüklenmesi: https://www.microsoft.com/en-us/download/details.aspx?id=57467

2) Visual Studio C/C++ araçları ile kurulumu

3) Açılan Projede yapılması gereken ayarlamalar:

- Proje/Özellikler/C/C++/Ek İçeren Dizinler → Yeni bir dizin ekleme: \Microsoft SDKs\MPI\Include
- Proje/Özellikler/Bağlayıcı/Ek Kitaplık Dizinleri → Yeni bir dizin ekleme: Microsoft SDKs\MPI\Lib\x86-x64
- Proje/Özellikler/Bağlayıcı/Giriş/Ek Bağımlılıklar → Yeni bir bağımlılık ekleme: msmpi.lib

Seri «Hello World» Uygulaması

```
#include <stdio.h>
int main(void) {
    printf("hello, world\n");

    return 0;
}
```

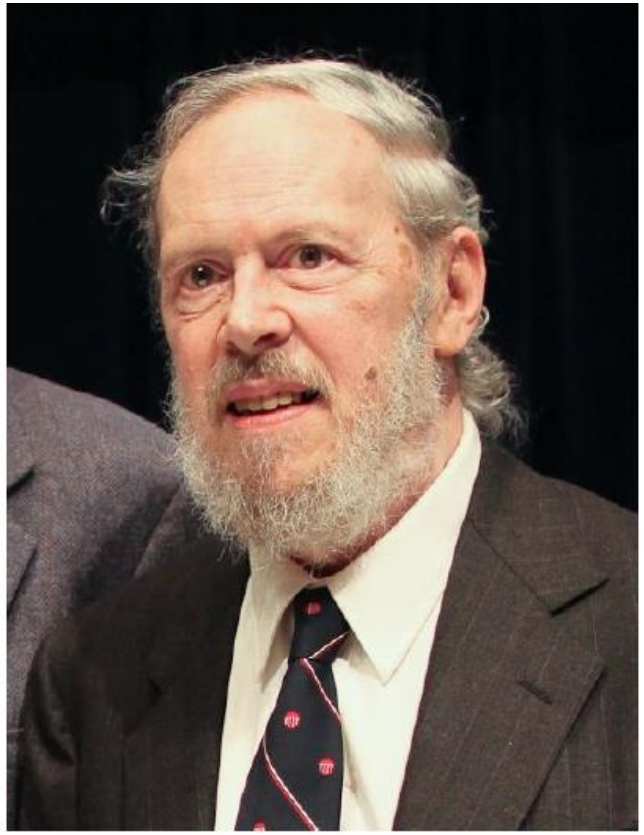
Brian Wilson Kernighan
The "K" of K&R C Book
Father of Unix
Creator of AWK and AMPL
programming languages

Şimdi bu programı MPI kullanarak biraz daha değiştirelim

MPI ile «Hello World» Uygulaması

```
#include <stdio.h>
#include <mpi.h>

main(int argc, char *argv[])
{
        int npes, myrank;
        MPI_Init(&argc, &argv);
        MPI_Comm_size(MPI_COMM_WORLD, &npes);
        MPI_Comm_rank(MPI_COMM_WORLD, &myrank);
        printf("From process %d out of %d, Hello
               World!\n", myrank, npes);
        MPI_Finalize();
}
```



Dennis Ritchie
The Father of C
And the Father of Unix

MPI ile «Hello World» Uygulaması 2

```c
#include <stdio.h>
#include <string.h>  /* For strlen             */
#include <mpi.h>     /* For MPI functions, etc */
const int MAX_STRING = 100;
int main(void) {
    char       greeting[MAX_STRING];
    int        comm_sz; // Number of Process
    int        my_rank; // id of the Process
    /* Start up MPI */
    MPI_Init(NULL, NULL);
    /* Get the number of processes */
    MPI_Comm_size(MPI_COMM_WORLD, &comm_sz);
    /* Get my rank among all the processes */
    MPI_Comm_rank(MPI_COMM_WORLD, &my_rank);
    if (my_rank != 0) {
        sprintf_s(greeting, "%d Slave process'den merhaba, toplam %d process var!", my_rank, comm_sz);
        MPI_Send(greeting, strlen(greeting) + 1, MPI_CHAR, 0, 0, MPI_COMM_WORLD);
        }
    else {
        printf("%d nolu Master process'den selamlar, toplam %d process var!\n", my_rank, comm_sz);
        for (int q = 1; q < comm_sz; q++){
            MPI_Recv(greeting, MAX_STRING, MPI_CHAR, q, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
            printf("%s\n", greeting);
        }
    }
    /* Shut down MPI */
    MPI_Finalize();
    return 0;
}  /* main */
```

MPI Programını Çalıştırmak

- Başarılı bir şekilde derlenen programı çalıştırmak için komut istemi üzerinden Dosyaismi.exe'nin olduğu klasöre giderek yine cmd üzerinden:

mpiexec –n 4 Dosyaismi.exe (ya da –n yerine -np)

komutu çalıştırılmalıdır.

- Burada 4 değeri paralel olarak çalıştırılacak olan process sayısını ifade etmektedir. Program çıktısı:

```
Greetings from master process 0 of 4
Greetings from slave process 1 of 4
Greetings from slave process 2 of 4
Greetings from slave process 3 of 4
```

MPI Programına Derinlemesine Bakış

- include <mpi.h> ile programımıza MPI kütüphanesi import edilir. MPI programlarının tamamında bu Header dosyası mutlaka eklenmelidir.
- Bu kütüphane içerisinde MPI kütüphanesine ait fonksiyonların, global değişkenlerin, makroların tanımlamaları saklanmaktadır.
- MPI kütüphanesine ait tüm fonksiyon ve değişkenler MPI_ dizgesi ile başlamaktadır. Bu yapılara MPI-tanımlı türler (MPI-defined types) denmektedir.

MPI_Init ve MPI_Finalize

- MPI_Init sisteme MPI ile ilgili bütün kurulumları hazırlamasını söyler.
- MPI_Init 'den önce hiçbir MPI fonksiyonu çağırılamaz ve kullanılamaz.

```
int MPI_Init(
        int* argc_p /* in/out */,
        char*** argv_p /* in/out */);
```

- Bu fonksiyonlar main ile kullanılan argc ve argv argümanlarına karşılık gelirler ve kullanılmadıklarında zaman NULL olarak tanımlanmalıdırlar.

MPI_Init ve MPI_Finalize

- Birçok MPI fonksiyonu gibi MPI_Init de sisteme int tipinde bir hata kodu gönderir. Bu hata kodları genellikle ihmal edilebilirler.
- int MPI_Finalize(void) ; Bu fonksiyon ise MPI kütüphanesinin işini bitirdiğini ifade eder.
- Ayrıca hafızada MPI ile ilgili ayrılan bütün alanların serbest bırakılmasını sağlar.
- Genellikle MPI_Finalize fonksiyonundan sonra herhangi bir MPI fonksiyonu kullanılmaz.

MPI_Comm_size ve MPI_rank

- MPI içerisinde tanımlanan işlemler(processler) birbirlerine mesaj gönderebilirler.
- MPI kütüphanesinde işlemler arasındaki iletişimi MPI_COMM_WORLD adı verilen global bir argüman ile sağlar.
- Bunun içerisinde aşağıdaki gibi tutulmaktadırlar.

```
int MPI_Comm_size(
     MPI_Comm comm /* in */,
     int*     comm_size /* out */);
int MPI_Comm_rank(
     MPI_Comm comm /* in */,
     int*     my_rank /* out */);
```

MPI_Comm_size ve MPI_rank

- MPI_Comm_Size fonksiyonu iletişim yapısından (struct) toplam işlem (process) sayısını alır.
- MPI_Comm_rank fonksiyonu ise iletişim yapısından (struct) işlemin rank (id) değerini geri döndürür.
- Bu rutinlerin çağrı sıraları aşağıdaki gibidir:

```
int MPI_Comm_size(MPI_Comm comm, int *size)
int MPI_Comm_rank(MPI_Comm comm, int *rank)
```

MPI_Send ve MPI_Recv

- MPI'da mesaj gönderip almak için kullanılan temel fonksiyonlar MPI_Send ve MPI_Recv fonksiyonlarıdır.
- Bu fonksiyonların çağrı sıraları şu şekildedir:

```
int MPI_Send(void *buf, int size, MPI_Datatype
      datatype, int dest, int tag, MPI_Comm comm)
int MPI_Recv(void *buf, int size, MPI_Datatype
      datatype, int source, int tag, MPI_Comm comm,
      MPI_Status *status)
```

- MPI, tüm C veri türleri için eşdeğer veri türleri sağlar. Bu taşınabilirlik amacı için yapılmaktadır.

Bazı Tanımlı MPI Veri Türleri

MPI Datatype	C Datatype
MPI_CHAR	signed char
MPI_SHORT	signed short int
MPI_INT	signed int
MPI_LONG	signed long int
MPI_UNSIGNED_CHAR	unsigned char
MPI_UNSIGNED_SHORT	unsigned short int
MPI_UNSIGNED	unsigned int
MPI_UNSIGNED_LONG	unsigned long int
MPI_FLOAT	float
MPI_DOUBLE	double
MPI_LONG_DOUBLE	long double
MPI_BYTE	
MPI_PACKED	

Örn: MPI_Send

```
MPI_Send(greeting, strlen(greeting)+1, MPI_CHAR, 0, 0,
         MPI_COMM_WORLD);
```

- Buradaki fonsiyonda:
  - greeting — message_buffer*,
  - strlen(greeting)+1 — size,
  - MPI_CHAR — datatype,
  - 0 — dest,
  - 0 — tag,
  - MPI_COMM_WORLD — MPI_Comm,

olarak ifade edilmiştir.

- Burada size değeri değişken olabilir fakat MAX_STRING değerinden büyük olamaz.

Örn: MPI_Recv

```
MPI_Recv(greeting, MAX_STRING, MPI_CHAR, q, 0,
         MPI_COMM_WORLD, MPI_STATUS_IGNORE);
```

- Buradaki fonsiyonda:
  - greeting — message_buffer*,
  - MAX_STRING — size,
  - MPI_CHAR — datatype,
  - q — source,
  - 0 — tag,
  - MPI_COMM_WORLD — MPI_Comm,
  - MPI_STATUS_IGNORE — MPI_Status*,

olarak ifade edilmiştir.

Bknz:

https://mpitutorial.com/tutorials/

MPI_Status Yapısı

- Veri alımı sonunda, durum değişkeni (`MPI_Status*`), `MPI_Recv` işlemi hakkında bilgi almak için kullanılabilir.
- İlgili veri yapısı şunları içerir:

```
typedef struct MPI_Status {
        int MPI_SOURCE;
        int MPI_TAG;
        int MPI_ERROR; };
```

Teşekkürler.

Sorular...